SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

QUESTIONS Total Marks: 50

Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	**5**
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	**5**
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	**5**
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	**5**

Code of Conduct:

*I __**S.Jaya Shree(192524146)**_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.Jaya Shree

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q1. Online Shopping Platform – Functional Dependency Analysis and 3NF

1. Given Relation

The online shopping platform maintains the following relation:
Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)
Assumption: Each OrderID represents one order containing one product.

2. Case Analysis

The relation combines three different types of information:

- **Order information** – OrderID and Qty
- **Customer information** – CustID and CustName
- **Product information** – ProdID, ProdName and Price

Storing all these details in one relation can cause the same customer and product information to appear repeatedly. For example, whenever a customer places another order, their name may be stored again. Similarly, the product name and price may be repeated for every order containing that product.
This repetition increases storage requirements and can cause inconsistencies when information is modified.

3. Functional Dependency Identification

The functional dependencies are determined from the meaning of each attribute.

Determinant	**Dependent Attributes**	**Reason**
OrderID	CustID, ProdID, Qty	An order identifies its customer, product and quantity
CustID	CustName	Each customer ID identifies one customer
ProdID	ProdName, Price	Each product ID determines its product details

Therefore:
FD Set:
OrderID → CustID, ProdID, Qty
CustID → CustName
ProdID → ProdName, Price
Since an order contains one product under our assumption:
Candidate Key = OrderID

4. Conversion to First Normal Form (1NF)

A relation satisfies 1NF when every attribute contains a single, indivisible value.
In the given relation:

- OrderID contains one order identifier.
- CustID contains one customer identifier.

- ProdID contains one product identifier.
- Qty and Price contain single values.
- There are no repeating groups.

Hence, the given relation satisfies **1NF**.

1NF Relation

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

5. Conversion to Second Normal Form (2NF)

The candidate key is OrderID, which consists of only **one attribute**.

Partial dependency can occur only when a candidate key contains multiple attributes. Since the key is not composite, there is no possibility of a partial dependency.

Therefore:

The relation is in 2NF.

6. Conversion to Third Normal Form (3NF)

Although the relation is in 2NF, it contains **transitive dependencies**.

The dependencies can be represented as:

OrderID → CustID → CustName

and

OrderID → ProdID → ProdName, Price

Here, CustName, ProdName, and Price do not directly depend on the order identifier. They depend on other non-key attributes.

To eliminate these transitive dependencies, the relation is divided into three relations.

7. Decomposition into 3NF

A. Customer Relation

Customer(CustID, CustName)

Here:

CustID → CustName

Primary Key: CustID

B. Product Relation

Product(ProdID, ProdName, Price)

Here:

ProdID → ProdName, Price

Primary Key: ProdID

C. Order Relation

Order(OrderID, CustID, ProdID, Qty)

Here:

OrderID → CustID, ProdID, Qty

Primary Key: OrderID

Foreign Keys:

- CustID references Customer(CustID)

- ProdID references Product(ProdID)

8. Problems in the Original Relation

Update Anomaly

If a product price changes, several order records may need to be modified. Missing even one update can produce inconsistent prices.

Insertion Anomaly

A new product cannot be conveniently stored without creating an order record if the original table is used.

Deletion Anomaly

If the only order containing a particular product is deleted, information about that product may also disappear.

Advantages of the 3NF Design

The normalized design provides:

1. Reduced duplication of customer and product details.
2. Better consistency of stored information.
3. Easier modification of customer and product records.
4. Independent insertion of customers and products.
5. Protection against accidental loss of master data.
6. Proper relationships through primary and foreign keys.
7. Improved database maintainability.

10. Final Result

The given online shopping relation was analyzed using functional dependencies and normalized systematically. Since OrderID is a single-attribute candidate key, the relation directly satisfies **2NF** after satisfying 1NF. Transitive dependencies are then removed by separating customer and product information. The resulting **Customer, Product, and Order** relations satisfy **3NF** and provide a structured database with reduced redundancy and improved data integrity.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input

mysql> CREATE DATABASE OnlineShopping;
Query OK, 1 row affected (0.05 sec)

mysql> USE OnlineShopping;
Database changed
mysql>
mysql> -- Customer table
mysql> CREATE TABLE Customer (
    ->     CustID INT PRIMARY KEY,
    ->     CustName VARCHAR(50) NOT NULL
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> -- Product table
mysql> CREATE TABLE Product (
    ->     ProdID INT PRIMARY KEY,
    ->     ProdName VARCHAR(100) NOT NULL,
    ->     Price DECIMAL(10,2) NOT NULL
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> -- Order table
mysql> CREATE TABLE Orders (
    ->     OrderID INT PRIMARY KEY,
    ->     CustID INT,
    ->     ProdID INT,
    ->     Qty INT NOT NULL,
    ->     FOREIGN KEY (CustID) REFERENCES Customer(CustID),
    ->     FOREIGN KEY (ProdID) REFERENCES Product(ProdID)
```

```
mysql> -- Insert customer data
mysql> INSERT INTO Customer VALUES
    -> (101, 'Arun'),
    -> (102, 'Priya'),
    -> (103, 'Rahul');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Insert product data
mysql> INSERT INTO Product VALUES
    -> (201, 'Laptop', 55000.00),
    -> (202, 'Keyboard', 1500.00),
    -> (203, 'Mouse', 800.00);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Insert order data
mysql> INSERT INTO Orders VALUES
    -> (1001, 101, 201, 1),
    -> (1002, 102, 202, 2),
    -> (1003, 103, 203, 3);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Display Customer
mysql> SELECT * FROM Customer;
+---------+-----------+
| CustID | CustName |
+---------+-----------+
|     101 | Arun      |
|     102 | Priya     |
|     103 | Rahul     |
+---------+-----------+
3 rows in set (0.00 sec)
```

```
mysql> -- Display complete order details
mysql> SELECT
    ->     O.OrderID,
    ->     C.CustName,
    ->     P.ProdName,
    ->     O.Qty,
    ->     P.Price
    -> FROM Orders O
    -> JOIN Customer C ON O.CustID = C.CustID
    -> JOIN Product P ON O.ProdID = P.ProdID;
+---------+----------+----------+-----+----------+
| OrderID | CustName | ProdName | Qty | Price    |
+---------+----------+----------+-----+----------+
|    1001 | Arun     | Laptop   |   1 | 55000.00 |
|    1002 | Priya    | Keyboard |   2 |  1500.00 |
|    1003 | Rahul    | Mouse    |   3 |   800.00 |
+---------+----------+----------+-----+----------+
3 rows in set (0.01 sec)

mysql>
mysql>
```

Q2. Hospital Patient Management – Normalization to BCNF

1. Given Relation

The hospital maintains the following relation:

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Assumption: Each patient is assigned to one doctor and one ward. Each DocID uniquely identifies a doctor, and each WardNo uniquely identifies a ward.

2. Case Analysis

The relation stores **patient, doctor, and ward information** in one table. When several patients are treated by the same doctor or stay in the same ward, doctor and ward information is repeated.

For example, the same doctor's name and specialization may appear in many patient records. Similarly, the same ward name may be repeated for every patient assigned to that ward.

This creates unnecessary redundancy and can result in **update, insertion, and deletion anomalies**.

3. Functional Dependencies

The important functional dependencies are:

No.	Functional Dependency	Meaning
1	PID → PName, DocID, WardNo	Patient ID determines patient details
2	DocID → DocName, Specialization	Doctor ID determines doctor details
3	WardNo → WardName	Ward number determines ward name

Therefore:

Candidate Key = PID

4. First Normal Form (1NF)

The relation contains atomic values and no repeating groups.

Therefore, the relation satisfies **1NF**.

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

5. Second Normal Form (2NF)

The candidate key is PID, which is a single attribute.

Therefore, partial dependency cannot occur.

Hence, the relation is in **2NF**.

6. Third Normal Form (3NF)

There are transitive dependencies:

PID → DocID → DocName, Specialization

and

PID → WardNo → WardName

Therefore, doctor and ward details should be separated from patient information.

7. BCNF Decomposition

Patient Table

Patient(PID, PName, DocID, WardNo)

Primary Key: PID

Foreign Keys:

- DocID
- WardNo

Doctor Table

Doctor(DocID, DocName, Specialization)

Primary Key: DocID

Ward Table

Ward(WardNo, WardName)

Primary Key: WardNo

8 Anomalies in the Original Relation

Update Anomaly

If a doctor's specialization changes, the information may have to be changed in several patient records.

Insertion Anomaly

A new doctor cannot be stored independently if the table requires a patient record.

Deletion Anomaly

If the last patient assigned to a doctor is deleted, the doctor's information may also be lost.

9. BCNF Verification

A relation is in **BCNF** if every determinant is a candidate key.

Patient

PID → PName, DocID, WardNo

PID is the primary key.

Doctor

DocID → DocName, Specialization

DocID is the primary key.

Ward

WardNo → WardName

WardNo is the primary key.

Therefore, all determinants are candidate keys.

Hence, the decomposed relations satisfy BCNF.

```
mysql>
mysql> -- Q2: Hospital Patient Management System
mysql> -- Normalization up to BCNF
mysql>
mysql> CREATE DATABASE HospitalDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE HospitalDB;
Database changed
mysql>
mysql> -- Doctor table
mysql> CREATE TABLE Doctor (
    ->     DocID INT PRIMARY KEY,
    ->     DocName VARCHAR(50) NOT NULL,
    ->     Specialization VARCHAR(50) NOT NULL
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> -- Ward table
mysql> CREATE TABLE Ward (
    ->     WardNo INT PRIMARY KEY,
    ->     WardName VARCHAR(50) NOT NULL
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> -- Patient table
mysql> CREATE TABLE Patient (
    ->     PID INT PRIMARY KEY,
    ->     PName VARCHAR(50) NOT NULL,
    ->     DocID INT,
    ->     WardNo INT,
    ->     FOREIGN KEY (DocID) REFERENCES Doctor(DocID),
    ->     FOREIGN KEY (WardNo) REFERENCES Ward(WardNo)
```

```
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> -- Insert doctor records
mysql> INSERT INTO Doctor VALUES
    -> (101, 'Dr. Kumar', 'Cardiology'),
    -> (102, 'Dr. Priya', 'Neurology'),
    -> (103, 'Dr. Ravi', 'Orthopedics');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Insert ward records
mysql> INSERT INTO Ward VALUES
    -> (1, 'General Ward'),
    -> (2, 'ICU'),
    -> (3, 'Special Ward');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Insert patient records
mysql> INSERT INTO Patient VALUES
    -> (1001, 'Arun', 101, 1),
    -> (1002, 'Priya', 102, 2),
    -> (1003, 'Rahul', 103, 3);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Display Doctor table
mysql> SELECT * FROM Doctor;
+-------+-----------+----------------+
| DocID | DocName   | Specialization |
+-------+-----------+----------------+
|   101 | Dr. Kumar | Cardiology     |
```

```
mysql> SELECT * FROM Doctor;
+-------+-----------+----------------+
| DocID | DocName   | Specialization |
+-------+-----------+----------------+
|   101 | Dr. Kumar | Cardiology     |
|   102 | Dr. Priya | Neurology      |
|   103 | Dr. Ravi  | Orthopedics    |
+-------+-----------+----------------+
3 rows in set (0.00 sec)

mysql>
mysql> -- Display Ward table
mysql> SELECT * FROM Ward;
+--------+-------------+
| WardNo | WardName    |
+--------+-------------+
|      1 | General Ward |
|      2 | ICU          |
|      3 | Special Ward |
+--------+-------------+
3 rows in set (0.00 sec)

mysql>
mysql> -- Display Patient table
mysql> SELECT * FROM Patient;
+------+-------+-------+--------+
| PID  | PName | DocID | WardNo |
+------+-------+-------+--------+
| 1001 | Arun  |   101 |      1 |
| 1002 | Priya |   102 |      2 |
| 1003 | Rahul |   103 |      3 |
+------+-------+-------+--------+
3 rows in set (0.00 sec)

mysql>
mysql> -- Display complete hospital information
mysql> SELECT
```

```
mysql>
mysql> -- Display Patient table
mysql> SELECT * FROM Patient;
+------+-------+-------+--------+
| PID  | PName | DocID | WardNo |
+------+-------+-------+--------+
| 1001 | Arun  |   101 |      1 |
| 1002 | Priya |   102 |      2 |
| 1003 | Rahul |   103 |      3 |
+------+-------+-------+--------+
3 rows in set (0.00 sec)

mysql>
mysql> -- Display complete hospital information
mysql> SELECT
    ->     P.PID,
    ->     P.PName,
    ->     D.DocName,
    ->     D.Specialization,
    ->     W.WardName
    -> FROM Patient P
    -> JOIN Doctor D
    ->     ON P.DocID = D.DocID
    -> JOIN Ward W
    ->     ON P.WardNo = W.WardNo;
+------+-------+----------+----------------+--------------+
| PID  | PName | DocName  | Specialization | WardName     |
+------+-------+----------+----------------+--------------+
| 1001 | Arun  | Dr. Kumar | Cardiology     | General Ward |
| 1002 | Priya | Dr. Priya | Neurology      | ICU          |
| 1003 | Rahul | Dr. Ravi  | Orthopedics    | Special Ward |
+------+-------+----------+----------------+--------------+
3 rows in set (0.00 sec)

mysql>
```

Result

The hospital relation was successfully decomposed into Patient, Doctor, and Ward relations. The decomposition eliminates transitive dependencies and redundancy, and each resulting relation satisfies BCNF under the stated assumptions.

Q3. University Enrollment – 2NF Decomposition

1. Given Relation

The university stores enrollment information in the following relation:
Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)
Assumption: A student can enroll in multiple courses, and a course can have multiple students.

2. Case Analysis

The relation contains both **student details** and **course details** along with the student's grade. When a student enrolls in several courses, the student's name is repeated. Similarly, the course name and faculty are repeated for every student taking that course.
This creates unnecessary duplication and may result in update, insertion, and deletion anomalies.

3. Functional Dependencies

The functional dependencies are:

Determinant	**Dependent Attributes**	**Explanation**
SID	SName	Student ID identifies the student
CourseID	CourseName, Faculty	Course ID identifies course details
(SID, CourseID)	Grade	Grade depends on a particular student-course combination

Therefore, the candidate key is:
Candidate Key = (SID, CourseID)

4. First Normal Form (1NF)

The relation contains atomic values and does not contain repeating groups.
Therefore:
Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)
is in **1NF**.

5. Second Normal Form (2NF)

The candidate key is composite:
(SID, CourseID)
However, some attributes depend only on part of the composite key:
SID → SName
CourseID → CourseName, Faculty
These are **partial dependencies**.
Therefore, the original relation is **not in 2NF**.

6. 2NF Decomposition

To remove the partial dependencies, divide the relation into three tables.

Student

Student(SID, SName)

Primary Key: SID

Course

Course(CourseID, CourseName, Faculty)

Primary Key: CourseID

Enrollment

Enrollment(SID, CourseID, Grade)

Primary Key: (SID, CourseID)

Foreign Keys:

- SID → Student(SID)
- CourseID → Course(CourseID)

7. Advantages of Decomposition

Removes partial dependencies.

- Reduces repeated student information.
- Reduces repeated course information.
- Makes updates easier.
- Prevents unnecessary data duplication.
- Maintains relationships using foreign keys.

```
mysql> -- Q3: University Enrollment
mysql> -- Normalization up to 2NF
mysql>
mysql> CREATE DATABASE UniversityDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE UniversityDB;
Database changed
mysql>
mysql> -- Student table
mysql> CREATE TABLE Student (
    ->     SID INT PRIMARY KEY,
    ->     SName VARCHAR(50) NOT NULL
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> -- Course table
mysql> CREATE TABLE Course (
    ->     CourseID INT PRIMARY KEY,
    ->     CourseName VARCHAR(100) NOT NULL,
    ->     Faculty VARCHAR(50) NOT NULL
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> -- Enrollment table
mysql> CREATE TABLE Enrollment (
    ->     SID INT,
    ->     CourseID INT,
    ->     Grade VARCHAR(5),
    ->     PRIMARY KEY (SID, CourseID),
    ->     FOREIGN KEY (SID) REFERENCES Student(SID),
    ->     FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
    -> );
```

```
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> -- Insert student records
mysql> INSERT INTO Student VALUES
    -> (101, 'Arun'),
    -> (102, 'Priya'),
    -> (103, 'Rahul');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Insert course records
mysql> INSERT INTO Course VALUES
    -> (201, 'Database Management System', 'Dr. Kumar'),
    -> (202, 'Data Structures', 'Dr. Priya'),
    -> (203, 'Computer Networks', 'Dr. Ravi');
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Insert enrollment records
mysql> INSERT INTO Enrollment VALUES
    -> (101, 201, 'A'),
    -> (101, 202, 'B+'),
    -> (102, 201, 'A+'),
    -> (102, 203, 'A'),
    -> (103, 202, 'B');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Display Student table
mysql> SELECT * FROM Student;
+-----+-------+
| SID | SName |
+-----+-------+
```

```
+-----+-------+
| SID | SName |
+-----+-------+
| 101 | Arun  |
| 102 | Priya |
| 103 | Rahul |
+-----+-------+
3 rows in set (0.00 sec)

mysql>
mysql> -- Display Course table
mysql> SELECT * FROM Course;
+----------+---------------------------+-----------+
| CourseID | CourseName                | Faculty   |
+----------+---------------------------+-----------+
|      201 | Database Management System | Dr. Kumar |
|      202 | Data Structures           | Dr. Priya |
|      203 | Computer Networks         | Dr. Ravi  |
+----------+---------------------------+-----------+
3 rows in set (0.00 sec)

mysql>
mysql> -- Display Enrollment table
mysql> SELECT * FROM Enrollment;
+-----+----------+-------+
| SID | CourseID | Grade |
+-----+----------+-------+
| 101 |      201 | A     |
| 101 |      202 | B+    |
| 102 |      201 | A+    |
| 102 |      203 | A     |
| 103 |      202 | B     |
+-----+----------+-------+
5 rows in set (0.00 sec)

mysql>
mysql> -- Display complete enrollment details
```

```
5 rows in set (0.00 sec)

mysql>
mysql> -- Display complete enrollment details
mysql> SELECT
    ->     E.SID,
    ->     S.SName,
    ->     E.CourseID,
    ->     C.CourseName,
    ->     C.Faculty,
    ->     E.Grade
    -> FROM Enrollment E
    -> JOIN Student S
    ->     ON E.SID = S.SID
    -> JOIN Course C
    ->     ON E.CourseID = C.CourseID;
+-----+-------+----------+---------------------------+-----------+-------+
| SID | SName | CourseID | CourseName                | Faculty   | Grade |
+-----+-------+----------+---------------------------+-----------+-------+
| 101 | Arun  |      201 | Database Management System | Dr. Kumar | A     |
| 101 | Arun  |      202 | Data Structures           | Dr. Priya | B+    |
| 102 | Priya |      201 | Database Management System | Dr. Kumar | A+    |
| 102 | Priya |      203 | Computer Networks         | Dr. Ravi  | A     |
| 103 | Rahul |      202 | Data Structures           | Dr. Priya | B     |
+-----+-------+----------+---------------------------+-----------+-------+
5 rows in set (0.00 sec)

mysql>
mysql>
```

Result

The University Enrollment relation was successfully analyzed and decomposed from **1NF to 2NF**. Partial dependencies were removed by separating student, course, and enrollment information into independent relations.

Q4. Airline Booking – Normalization up to BCNF

1. Given Relation

The airline booking system stores:

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Assumption: A passenger can book a particular flight on a particular date, and each seat on a flight/date combination is assigned to at most one passenger.

2. Case Analysis

The relation combines **flight information, passenger information, and booking information** in one table.

If many passengers travel on the same flight, the departure and arrival cities are repeated. Similarly, the passenger's name is repeated whenever the passenger makes multiple bookings. This causes unnecessary redundancy and may result in update, insertion, and deletion anomalies.

3. Functional Dependencies

The important functional dependencies are:

Determinant	**Dependent Attributes**	**Explanation**
PassID	PassName	Passenger ID identifies passenger name
FlightNo	DeptCity, ArrCity	Flight number identifies its route
(FlightNo, Date, PassID)	Seat	A passenger's booking determines the assigned seat

Therefore, the candidate key is:

Candidate Key = (FlightNo, Date, PassID)

4. First Normal Form (1NF)

All attributes contain atomic values and there are no repeating groups.

Therefore, the relation is in **1NF**.

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

5. Second Normal Form (2NF)

The candidate key is composite:

(FlightNo, Date, PassID)

Partial dependencies exist:

PassID → PassName

FlightNo → DeptCity, ArrCity

Therefore, the original relation is **not in 2NF**.

6. 2NF Decomposition

Passenger

Passenger(PassID, PassName)

Primary Key: PassID

Flight

Flight(FlightNo, DeptCity, ArrCity)

Primary Key: FlightNo

Booking

Booking(FlightNo, Date, PassID, Seat)

Primary Key: (FlightNo, Date, PassID)

7. BCNF Verification

A relation is in BCNF when every determinant is a candidate key.

Passenger

PassID → PassName

PassID is the primary key.

Therefore, it satisfies BCNF.

Flight

FlightNo → DeptCity, ArrCity

FlightNo is the primary key.

Therefore, it satisfies BCNF.

Booking

(FlightNo, Date, PassID) → Seat

The complete composite key determines the seat.

Therefore, under the stated assumptions, it satisfies BCNF.

9. Anomalies Removed

Update Anomaly

Changing a passenger's name requires modification in only the Passenger table.

Insertion Anomaly

A new passenger or flight can be stored without requiring a booking.

Deletion Anomaly

Deleting a booking does not remove the passenger or flight information.

```
mysql> -- Q4: Airline Booking
mysql> -- Normalization up to BCNF
mysql>
mysql> CREATE DATABASE AirlineDB;
Query OK, 1 row affected (0.03 sec)

mysql> USE AirlineDB;
Database changed
mysql>
mysql> -- Passenger table
mysql> CREATE TABLE Passenger (
    ->     PassID INT PRIMARY KEY,
    ->     PassName VARCHAR(50) NOT NULL
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> -- Flight table
mysql> CREATE TABLE Flight (
    ->     FlightNo VARCHAR(10) PRIMARY KEY,
    ->     DeptCity VARCHAR(50) NOT NULL,
    ->     ArrCity VARCHAR(50) NOT NULL
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> -- Booking table
mysql> CREATE TABLE Booking (
    ->     FlightNo VARCHAR(10),
    ->     Date DATE,
    ->     PassID INT,
    ->     Seat VARCHAR(10),
    ->     PRIMARY KEY (FlightNo, Date, PassID),
    ->     FOREIGN KEY (FlightNo) REFERENCES Flight(FlightNo),
    ->     FOREIGN KEY (PassID) REFERENCES Passenger(PassID)
    -> );
```

```
mysql>
mysql> -- Insert passenger records
mysql> INSERT INTO Passenger VALUES
    -> (101, 'Arun'),
    -> (102, 'Priya'),
    -> (103, 'Rahul');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Insert flight records
mysql> INSERT INTO Flight VALUES
    -> ('AI101', 'Chennai', 'Delhi'),
    -> ('AI202', 'Mumbai', 'Bangalore'),
    -> ('AI303', 'Delhi', 'Kolkata');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Insert booking records
mysql> INSERT INTO Booking VALUES
    -> ('AI101', '2026-08-15', 101, '12A'),
    -> ('AI101', '2026-08-15', 102, '12B'),
    -> ('AI202', '2026-08-16', 103, '8C');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Display Passenger table
mysql> SELECT * FROM Passenger;
+--------+----------+
| PassID | PassName |
+--------+----------+
|    101 | Arun     |
|    102 | Priya    |
|    103 | Rahul    |
+--------+----------+
```

```
mysql>
mysql> -- Display complete booking details
mysql> SELECT
    ->     B.FlightNo,
    ->     B.Date,
    ->     P.PassID,
    ->     P.PassName,
    ->     B.Seat,
    ->     F.DeptCity,
    ->     F.ArrCity
    -> FROM Booking B
    -> JOIN Passenger P
    ->     ON B.PassID = P.PassID
    -> JOIN Flight F
    ->     ON B.FlightNo = F.FlightNo;
+----------+------------+--------+----------+------+----------+-----------+
| FlightNo | Date       | PassID | PassName | Seat | DeptCity | ArrCity   |
+----------+------------+--------+----------+------+----------+-----------+
| AI101    | 2026-08-15 |    101 | Arun     | 12A  | Chennai  | Delhi     |
| AI101    | 2026-08-15 |    102 | Priya    | 12B  | Chennai  | Delhi     |
| AI202    | 2026-08-16 |    103 | Rahul    | 8C   | Mumbai   | Bangalore |
+----------+------------+--------+----------+------+----------+-----------+
3 rows in set (0.01 sec)

mysql>
mysql>
```

Result

The airline booking relation was successfully decomposed into **Passenger, Flight, and Booking** relations. Partial dependencies were eliminated, and the resulting relations satisfy **BCNF** under the stated assumptions.

Q5. HR System – Functional and Transitive Dependencies and 3NF

1. Given Relation

For a multinational company's HR system, consider the following relation:
Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, LocationID, LocationName)
Assumption: Each employee belongs to one department, each department has one manager and one location, and each manager has a unique ManagerID.

2. Case Analysis

The relation contains employee, department, manager, and location information together. For a large organization, many employees may belong to the same department. Therefore, the same department name, manager name, and location details can be stored repeatedly.
This creates redundancy and can cause **update, insertion, and deletion anomalies**.

3. Functional Dependencies

The main functional dependencies are:

No.	Functional Dependency	Explanation
1	EmpID → EmpName, DeptID, ManagerID	Employee ID identifies employee details
2	DeptID → DeptName, LocationID	Department ID identifies department and its location
3	ManagerID → ManagerName	Manager ID identifies manager name
4	LocationID → LocationName	Location ID identifies location name

Therefore:
Candidate Key = EmpID

4. Transitive Dependencies

The relation contains several transitive dependencies.
Department Dependency
EmpID → DeptID
DeptID → DeptName
Therefore:
EmpID → DeptName
Location Dependency
EmpID → DeptID
DeptID → LocationID
LocationID → LocationName
Therefore:
EmpID → LocationName

Manager Dependency
EmpID → ManagerID
ManagerID → ManagerName
Therefore:
EmpID → ManagerName
These are transitive dependencies because non-key attributes depend on other non-key attributes.

5. First Normal Form (1NF)
All attributes contain atomic values and there are no repeating groups.
Therefore, the relation satisfies **1NF**.

6. Second Normal Form (2NF)
The candidate key is a single attribute:
EmpID
Since the key is not composite, partial dependency cannot occur.
Therefore, the relation is in **2NF**.

7. Third Normal Form (3NF)
The transitive dependencies must be removed.
The original relation is decomposed into the following relations.
Employee
Employee(EmpID, EmpName, DeptID, ManagerID)
Primary Key: EmpID
Department
Department(DeptID, DeptName, LocationID)
Primary Key: DeptID
Manager
Manager(ManagerID, ManagerName)
Primary Key: ManagerID
Location
Location(LocationID, LocationName)
Primary Key: LocationID

9. Anomalies Removed
Update Anomaly
Department or manager information can be updated in one place instead of multiple employee records.
Insertion Anomaly
A department, manager, or location can be added without first creating an employee.
Deletion Anomaly
Deleting an employee does not remove the associated department, manager, or location information.

```
mysql> CREATE DATABASE HRDB;
ERROR 1007 (HY000): Can't create database 'hrdb'; database exists
mysql> USE HRDB;
Database changed
mysql>
mysql> CREATE TABLE Manager(
    ->     ManagerID INT PRIMARY KEY,
    ->     ManagerName VARCHAR(30)
    -> );
ERROR 1050 (42S01): Table 'manager' already exists
mysql>
mysql> CREATE TABLE Department(
    ->     DeptID INT PRIMARY KEY,
    ->     DeptName VARCHAR(30),
    ->     ManagerID INT,
    ->     FOREIGN KEY(ManagerID) REFERENCES Manager(ManagerID)
    -> );
ERROR 1050 (42S01): Table 'department' already exists
mysql>
mysql> CREATE TABLE Employee(
    ->     EmpID INT PRIMARY KEY,
    ->     EmpName VARCHAR(30),
    ->     DeptID INT,
    ->     FOREIGN KEY(DeptID) REFERENCES Department(DeptID)
    -> );
ERROR 1050 (42S01): Table 'employee' already exists
mysql>
mysql> INSERT INTO Manager VALUES
    -> (1,'Raj'),(2,'Priya');
ERROR 1062 (23000): Duplicate entry '1' for key 'manager.PRIMARY'
mysql>
mysql> INSERT INTO Department VALUES
    -> (101,'HR',1),(102,'IT',2);
ERROR 1062 (23000): Duplicate entry '101' for key 'department.PRIMARY'
mysql>
mysql> INSERT INTO Employee VALUES
```

```
mysql> SELECT * FROM Employee;
+-------+---------+--------+
| EmpID | EmpName | DeptID |
+-------+---------+--------+
|  1001 | Arun    |    101 |
|  1002 | Rahul   |    102 |
+-------+---------+--------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Department;
+--------+----------+-----------+
| DeptID | DeptName | ManagerID |
+--------+----------+-----------+
|    101 | HR       |         1 |
|    102 | IT       |         2 |
+--------+----------+-----------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Manager;
+-----+--------+
| ID  | Name   |
+-----+--------+
|   1 | Raj    |
|   2 | Priya  |
| 501 | Rajesh |
| 502 | Priya  |
| 503 | Kumar  |
+-----+--------+
5 rows in set (0.00 sec)

mysql>
```

Result

The HR relation was analyzed by identifying its functional and transitive dependencies. The relation was decomposed into **Employee, Department, Manager, and Location** tables. The decomposition removes transitive dependencies and reduces redundancy, resulting in a **3NF database desig**

Q6. Library – Multivalued Dependencies and 4NF

1. Given Relation
Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)
Assumption: A member can borrow multiple books, and a book can have multiple authors. These independent multivalued facts should not be stored together.

2. Case Analysis
The original relation combines **member, book, and author information**. If a member borrows several books, their details may be repeated. If a book has multiple authors, combining both independent relationships can create unnecessary combinations of records.
This causes redundancy and may lead to insertion, deletion, and update anomalies.

3. Functional Dependencies
The main dependencies are:

- MemberID → MemberName
- BookID → Title, Genre
- BookID →→ Author
- (MemberID, BookID) → BorrowDate

The important **multivalued dependency (MVD)** is:
BookID →→ Author

4. 1NF
All attributes contain atomic values, so the relation satisfies **1NF**.

5. 2NF and 3NF
The member and book details depend on their respective identifiers. These dependencies create redundancy when stored in the same relation.
Separate the entities:
Member(MemberID, MemberName)
Book(BookID, Title, Genre)
Borrowing(MemberID, BookID, BorrowDate)
BookAuthor(BookID, Author)

6. 4NF Decomposition
The independent multivalued relationship between books and authors is separated into:
BookAuthor
BookAuthor(BookID, Author)
The borrowing relationship is stored separately:
Borrowing
Borrowing(MemberID, BookID, BorrowDate)

Final 4NF Schema

- Member(MemberID, MemberName)
- Book(BookID, Title, Genre)
- BookAuthor(BookID, Author)
- Borrowing(MemberID, BookID, BorrowDate)

```
mysql>
mysql> CREATE DATABASE LibraryDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE LibraryDB;
Database changed
mysql>
mysql> CREATE TABLE Member(
    ->     MemberID INT PRIMARY KEY,
    ->     MemberName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> CREATE TABLE Book(
    ->     BookID INT PRIMARY KEY,
    ->     Title VARCHAR(50),
    ->     Genre VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> CREATE TABLE BookAuthor(
    ->     BookID INT,
    ->     Author VARCHAR(30),
    ->     PRIMARY KEY(BookID,Author),
    ->     FOREIGN KEY(BookID) REFERENCES Book(BookID)
    -> );
Query OK, 0 rows affected (0.11 sec)

mysql>
mysql> CREATE TABLE Borrowing(
    ->     MemberID INT,
    ->     BookID INT,
    ->     BorrowDate DATE,
    ->     PRIMARY KEY(MemberID,BookID),
    ->     FOREIGN KEY(MemberID) REFERENCES Member(MemberID),
```

```
    ->     FOREIGN KEY(BookID) REFERENCES Book(BookID)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> INSERT INTO Member VALUES
    -> (1,'Arun'),(2,'Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Book VALUES
    -> (101,'DBMS','Education'),
    -> (102,'Python','Programming');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO BookAuthor VALUES
    -> (101,'Kumar'),(101,'Ravi'),(102,'John');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Borrowing VALUES
    -> (1,101,'2026-08-01'),
    -> (2,102,'2026-08-05');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Member;
+----------+------------+
| MemberID | MemberName |
+----------+------------+
|        1 | Arun       |
|        2 | Priya      |
```

```
mysql>
mysql> SELECT * FROM Member;
+----------+------------+
| MemberID | MemberName |
+----------+------------+
|        1 | Arun       |
|        2 | Priya      |
+----------+------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Book;
+--------+--------+-------------+
| BookID | Title  | Genre       |
+--------+--------+-------------+
|    101 | DBMS   | Education   |
|    102 | Python | Programming |
+--------+--------+-------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM BookAuthor;
+--------+--------+
| BookID | Author |
+--------+--------+
|    101 | Kumar  |
|    101 | Ravi   |
|    102 | John   |
+--------+--------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Borrowing;
+----------+--------+------------+
| MemberID | BookID | BorrowDate |
+----------+--------+------------+
|        1 |    101 | 2026-08-01 |
|        2 |    102 | 2026-08-05 |
+----------+--------+------------+
2 rows in set (0.00 sec)
```

Result

The library relation was decomposed into **Member, Book, BookAuthor, and Borrowing** relations. The multivalued dependency is separated, reducing redundant combinations and producing a **4NF design**.

Q7. Telecom Billing System – Keys and Functional Dependencies

1. Given Relation

Consider the telecom billing relation:

Billing(BillID, CustomerID, CustomerName, PlanID, PlanName, CallID, CallDuration, Rate, Amount)

2. Case Analysis

The relation stores customer, billing plan, call, and payment-related information together. Some attributes depend on the customer or plan rather than directly on the bill.

This can produce redundant data and anomalies.

3. Functional Dependencies

No.	Functional Dependency	Meaning
1	BillID → CustomerID, PlanID	Bill identifies customer and plan
2	CustomerID → CustomerName	Customer ID identifies customer
3	PlanID → PlanName, Rate	Plan ID identifies plan details
4	CallID → CallDuration	Call ID identifies call duration
5	(CallID, PlanID) → Amount	Amount depends on call and applicable plan

4. Candidate Key

For the billing relation:

Candidate Key = BillID

Since BillID uniquely identifies a bill, it can determine the bill-related information.

5. Primary Key

The suitable primary key is:

Primary Key = BillID

6. Transitive Dependencies

The relation contains transitive dependencies:

BillID → CustomerID → CustomerName

Therefore:

BillID → CustomerName

Similarly:

BillID → PlanID → PlanName, Rate

Therefore:

BillID → PlanName, Rate

These dependencies indicate that customer and plan information should be separated during normalization.

```
mysql>
mysql> CREATE DATABASE TelecomDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE TelecomDB;
Database changed
mysql>
mysql> CREATE TABLE Customer(
    ->     CustomerID INT PRIMARY KEY,
    ->     CustomerName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE Plan(
    ->     PlanID INT PRIMARY KEY,
    ->     PlanName VARCHAR(30),
    ->     Rate DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE Billing(
    ->     BillID INT PRIMARY KEY,
    ->     CustomerID INT,
    ->     PlanID INT,
    ->     Amount DECIMAL(10,2),
    ->     FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID),
    ->     FOREIGN KEY(PlanID) REFERENCES Plan(PlanID)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> INSERT INTO Customer VALUES
    -> (1,'Arun'),(2,'Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO Plan VALUES
    -> (101,'Basic',299),(102,'Premium',599);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Billing VALUES
    -> (1001,1,101,299),(1002,2,102,599);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Customer;
+------------+--------------+
| CustomerID | CustomerName |
+------------+--------------+
|          1 | Arun         |
|          2 | Priya        |
+------------+--------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Plan;
+--------+----------+--------+
| PlanID | PlanName | Rate   |
+--------+----------+--------+
|    101 | Basic    | 299.00 |
|    102 | Premium  | 599.00 |
+--------+----------+--------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Billing;
+--------+------------+--------+--------+
| BillID | CustomerID | PlanID | Amount |
+--------+------------+--------+--------+
|   1001 |          1 |    101 | 299.00 |
|   1002 |          2 |    102 | 599.00 |
+--------+------------+--------+--------+
```

8. Result

The telecom billing schema was analyzed to identify its **candidate key, primary key, and functional dependencies**. The analysis shows that customer and plan information can be maintained separately to reduce redundancy and improve data consistency.

8. Retail Inventory System – Complete Normalization

1. Given Relation
Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)
Assumption: Each product has one supplier and is stored in one warehouse.

2. Case Analysis
The relation stores both **product and supplier information** in the same table. If the same supplier provides multiple products, the supplier name is repeated for every product.
This creates unnecessary redundancy and may cause update, insertion, and deletion anomalies.

3. Functional Dependencies
The main functional dependencies are:

- PID → PName, SupplierID, Category, Price, Warehouse
- SupplierID → SupplierName

Therefore:
Candidate Key = PID

4. First Normal Form (1NF)
All attributes contain atomic values and there are no repeating groups.
Therefore, the relation is in **1NF**.

5. Second Normal Form (2NF)
The candidate key is a single attribute, PID.
Therefore, partial dependency cannot occur.
Hence, the relation is in **2NF**.

6. Third Normal Form (3NF)
There is a transitive dependency:
PID → SupplierID → SupplierName
Therefore, SupplierName should be separated from the Product relation.

7. BCNF / Complete Normalization
Supplier
Supplier(SupplierID, SupplierName)
Primary Key: SupplierID
Product
Product(PID, PName, SupplierID, Category, Price, Warehouse)
Primary Key: PID
Foreign Key: SupplierID
Both relations satisfy BCNF under the given assumptions because their determinants are candidate keys.

```
mysql>
mysql> CREATE DATABASE RetailDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE RetailDB;
Database changed
mysql>
mysql> CREATE TABLE Supplier(
    ->     SupplierID INT PRIMARY KEY,
    ->     SupplierName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE Product(
    ->     PID INT PRIMARY KEY,
    ->     PName VARCHAR(30),
    ->     SupplierID INT,
    ->     Category VARCHAR(30),
    ->     Price DECIMAL(10,2),
    ->     Warehouse VARCHAR(30),
    ->     FOREIGN KEY(SupplierID) REFERENCES Supplier(SupplierID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO Supplier VALUES
    -> (1,'ABC Suppliers'),
    -> (2,'XYZ Traders');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Product VALUES
    -> (101,'Laptop',1,'Electronics',55000,'Chennai'),
    -> (102,'Mouse',1,'Accessories',800,'Chennai'),
    -> (103,'Keyboard',2,'Accessories',1500,'Mumbai');
```

```
mysql>
mysql> INSERT INTO Product VALUES
    -> (101,'Laptop',1,'Electronics',55000,'Chennai'),
    -> (102,'Mouse',1,'Accessories',800,'Chennai'),
    -> (103,'Keyboard',2,'Accessories',1500,'Mumbai');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Supplier;
+------------+--------------+
| SupplierID | SupplierName |
+------------+--------------+
|          1 | ABC Suppliers |
|          2 | XYZ Traders  |
+------------+--------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Product;
+-----+---------+------------+------------+---------+-----------+
| PID | PName   | SupplierID | Category   | Price   | Warehouse |
+-----+---------+------------+------------+---------+-----------+
| 101 | Laptop  |          1 | Electronics | 55000.00 | Chennai   |
| 102 | Mouse   |          1 | Accessories |   800.00 | Chennai   |
| 103 | Keyboard |          2 | Accessories |  1500.00 | Mumbai    |
+-----+---------+------------+------------+---------+-----------+
3 rows in set (0.00 sec)

mysql>
```

Result

The retail inventory relation was normalized by removing the transitive dependency PID → SupplierID → SupplierName. The final **Supplier** and **Product** relations reduce redundancy and provide a normalized design up to **BCNF**.

Q9. School ERP – Normalization up to BCNF

1. Given Relation
Consider the poorly normalized School ERP relation:
School(SID, SName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Marks)
Assumption: Each student belongs to one class, each class has one teacher, and a student can study multiple subjects.

2. Case Analysis
The relation combines **student, class, teacher, subject, and marks information** in one table.
When a student studies multiple subjects, the same student and class information is repeated.
Similarly, teacher and subject information may be repeated across many records.
This creates data redundancy and can cause **insertion, deletion, and update anomalies**.

3. Functional Dependencies
The main functional dependencies are:

- SID → SName, ClassID
- ClassID → ClassName, TeacherID
- TeacherID → TeacherName
- SubjectID → SubjectName
- (SID, SubjectID) → Marks

Candidate Key
Since a student can have multiple subjects:
Candidate Key = (SID, SubjectID)

4. First Normal Form (1NF)
All attributes contain atomic values and there are no repeating groups.
Therefore, the relation is in **1NF**.

5. Second Normal Form (2NF)
The composite key is:
(SID, SubjectID)
Partial dependencies exist:
SID → SName, ClassID
SubjectID → SubjectName
Therefore, the original relation is **not in 2NF**.

6. Third Normal Form (3NF)
There are transitive dependencies:
SID → ClassID → ClassName, TeacherID
and:

ClassID → TeacherID → TeacherName
These dependencies must be separated.

7. BCNF Decomposition

Student
Student(SID, SName, ClassID)
Primary Key: SID
Class
Class(ClassID, ClassName, TeacherID)
Primary Key: ClassID
Teacher
Teacher(TeacherID, TeacherName)
Primary Key: TeacherID
Subject
Subject(SubjectID, SubjectName)
Primary Key: SubjectID
Marks
Marks(SID, SubjectID, Marks)
Primary Key: (SID, SubjectID)

```
mysql> CREATE DATABASE SchoolDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE SchoolDB;
Database changed
mysql>
mysql> CREATE TABLE Teacher(
    ->     TeacherID INT PRIMARY KEY,
    ->     TeacherName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE Class(
    ->     ClassID INT PRIMARY KEY,
    ->     ClassName VARCHAR(30),
    ->     TeacherID INT,
    ->     FOREIGN KEY(TeacherID) REFERENCES Teacher(TeacherID)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> CREATE TABLE Student(
    ->     SID INT PRIMARY KEY,
    ->     SName VARCHAR(30),
    ->     ClassID INT,
    ->     FOREIGN KEY(ClassID) REFERENCES Class(ClassID)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> CREATE TABLE Subject(
    ->     SubjectID INT PRIMARY KEY,
    ->     SubjectName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> INSERT INTO Teacher VALUES
    -> (1,'Kumar'),(2,'Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Class VALUES
    -> (101,'CSE-A',1),(102,'CSE-B',2);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Student VALUES
    -> (1001,'Arun',101),(1002,'Rahul',102);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Subject VALUES
    -> (201,'DBMS'),(202,'Python');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Marks VALUES
    -> (1001,201,85),(1001,202,90),(1002,201,78);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Student;
+------+-------+---------+
| SID  | SName | ClassID |
+------+-------+---------+
| 1001 | Arun  |     101 |
| 1002 | Rahul |     102 |
+------+-------+---------+
```

```
+---------+-----------+-----------+
| ClassID | ClassName | TeacherID |
+---------+-----------+-----------+
|     101 | CSE-A     |         1 |
|     102 | CSE-B     |         2 |
+---------+-----------+-----------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Teacher;
+-----------+-------------+
| TeacherID | TeacherName |
+-----------+-------------+
|         1 | Kumar       |
|         2 | Priya       |
+-----------+-------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Subject;
+-----------+-------------+
| SubjectID | SubjectName |
+-----------+-------------+
|       201 | DBMS        |
|       202 | Python      |
+-----------+-------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Marks;
+------+-----------+-------+
| SID  | SubjectID | Marks |
+------+-----------+-------+
| 1001 |       201 |    85 |
| 1001 |       202 |    90 |
| 1002 |       201 |    78 |
+------+-----------+-------+
3 rows in set (0.00 sec)

mysql>
```

Result

The School ERP relation was decomposed into **Student, Class, Teacher, Subject, and Marks** relations. Partial and transitive dependencies were removed, resulting in a structured design satisfying **BCNF** under the stated assumptions.

Q10. Movie Streaming Platform – Normalization to 5NF

1. Given Relation
Consider the following movie streaming relation:
Streaming(MovieID, ActorID, PlatformID)
Assumption: A movie can have multiple actors and can be available on multiple streaming platforms. These relationships are independent.

2. Case Analysis
The relation combines movie, actor, and streaming-platform relationships in one table.
For example, if one movie has 3 actors and is available on 2 platforms, storing all combinations can produce **6 rows**. This creates unnecessary repetition.
The problem is caused by a **join dependency**.

3. Join Dependency
The original relation can be represented using two independent relationships:
MovieActor(MovieID, ActorID)
MoviePlatform(MovieID, PlatformID)
The join dependency can be expressed as:
*(MovieID, ActorID), (MovieID, PlatformID)

4. 1NF
All attributes contain atomic values.
Therefore, the relation satisfies **1NF**.

5. 2NF, 3NF and 4NF
The relation contains independent relationships between:

- Movies and actors
- Movies and platforms

Separating these relationships eliminates unnecessary combinations.

6. 5NF Decomposition

MovieActor
MovieActor(MovieID, ActorID)
Primary Key: (MovieID, ActorID)
MoviePlatform
MoviePlatform(MovieID, PlatformID)
Primary Key: (MovieID, PlatformID)

Final 5NF Schema

MOVIE ACTOR

MovieID (PK)

ActorID (PK)

MOVIE PLATFORM

MovieID (PK)

PlatformID (PK)

7. Lossless-Join Verification

The original valid movie relationships can be reconstructed by joining the decomposed relations on MovieID.

MovieActor

|

| MovieID

↓

MoviePlatform

The decomposition is **lossless** when the business rule states that actor and platform relationships are independent.

```
mysql> CREATE DATABASE MovieDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE MovieDB;
Database changed
mysql>
mysql> CREATE TABLE MovieActor(
    ->      MovieID INT,
    ->      ActorID INT,
    ->      PRIMARY KEY(MovieID,ActorID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE MoviePlatform(
    ->      MovieID INT,
    ->      PlatformID INT,
    ->      PRIMARY KEY(MovieID,PlatformID)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> INSERT INTO MovieActor VALUES
    -> (1,101),(1,102),(2,103);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO MoviePlatform VALUES
    -> (1,201),(1,202),(2,201);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM MovieActor;
+----------+----------+
```

```
mysql> SELECT * FROM MovieActor;
+---------+---------+
| MovieID | ActorID |
+---------+---------+
|       1 |     101 |
|       1 |     102 |
|       2 |     103 |
+---------+---------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM MoviePlatform;
+---------+------------+
| MovieID | PlatformID |
+---------+------------+
|       1 |        201 |
|       1 |        202 |
|       2 |        201 |
+---------+------------+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT A.MovieID,A.ActorID,P.PlatformID
    -> FROM MovieActor A
    -> JOIN MoviePlatform P
    -> ON A.MovieID=P.MovieID;
+---------+---------+------------+
| MovieID | ActorID | PlatformID |
+---------+---------+------------+
|       1 |     101 |        201 |
|       1 |     101 |        202 |
|       1 |     102 |        201 |
|       1 |     102 |        202 |
|       2 |     103 |        201 |
+---------+---------+------------+
5 rows in set (0.00 sec)
```

Result

The movie streaming relation was decomposed into **MovieActor** and **MoviePlatform** relations to eliminate redundancy caused by the join dependency. The original information can be reconstructed through a lossless join under the stated independence assumption, giving a **5NF design**.